Imperial College London

COURSEWORK 2

IMPERIAL COLLEGE LONDON

DEPARTMENT OF MATHEMATICS

Computational Linear Algebra

Author:
(CID: 01844537)

Date: December 14, 2022

The Repository link is https://github.com/LarryWang29/CLA.

Question 1

1(a)

In this question, we apply the **"LUP_inplace"** algorithm from **"exercises6.py"** to the matrix $A^{(6)}$, which is defined as:

$$A^{(6)} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ -1 & 1 & 0 & 0 & 0 & 1 \\ -1 & -1 & 1 & 0 & 0 & 1 \\ -1 & -1 & -1 & 1 & 0 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & -1 & 1 \end{pmatrix} \tag{1}$$

After applying the LU-decomposition in place, we obtain that:

$$\mathrm{L} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 \\ -1 & -1 & 1 & 0 & 0 & 0 \\ -1 & -1 & -1 & 1 & 0 & 0 \\ -1 & -1 & -1 & -1 & 1 & 0 \\ -1 & -1 & -1 & -1 & -1 & 1 \end{pmatrix}, \mathrm{U} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 & 0 & 4 \\ 0 & 0 & 0 & 1 & 0 & 8 \\ 0 & 0 & 0 & 0 & 1 & 16 \\ 0 & 0 & 0 & 0 & 0 & 32 \end{pmatrix}$$

In the lecture notes, the growth factor ρ is defined as:

$$\rho = \frac{max_{ij} |u_{ij}|}{max_{ij} |a_{ij}|} \tag{2}$$

We can see that $max_{ij} |u_{ij}| = 32$, and $max_{ij} |a_{ij}| = 1$. Hence the growth factor is equal to 32.

1(b)

We claim that the general form for matrices $L^{(n)}$ and $U^{(n)}$ from the LUP-decomposition of $A^{(n)}$ are of the following form:

$$L^{(n)} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ -1 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \\ -1 & -1 & -1 & \cdots & 1 \end{pmatrix}, \qquad U^{(n)} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 1 \\ 0 & 1 & 0 & \cdots & 2 \\ 0 & 0 & 1 & \cdots & 4 \\ \vdots & \vdots & \vdots & \ddots & \\ 0 & 0 & 0 & \cdots & 2^{n-1} \end{pmatrix} \tag{3}$$

I.e.

$$L_{ij}^{(n)} = \begin{cases} 1 & \text{if } i = j \\ -1 & \text{if } i > j \\ 0 & \text{else} \end{cases}, \qquad U_{ij}^{(n)} = \begin{cases} 1 & \text{if } i = j \neq n \\ 2^{i-1} & \text{if } i = j = n \\ 0 & \text{else} \end{cases}$$

Proof:

We decompose $L^{(n)}$ and $U^{(n)}$ into blocks as such:

$$L^{(n)}U^{(n)} = \begin{pmatrix} L^{(n-1)} & 0 \\ -P^T & 1 \end{pmatrix} \begin{pmatrix} I & Q \\ 0 & 2^{n-1} \end{pmatrix}$$

where P is the length $n-1$ vector of 1's and Q is the vector $\begin{pmatrix} 1 & 2 & \cdots & 2^{n-2} \end{pmatrix}^T$. Multiplying out the blocks we obtain:

$$\begin{pmatrix} L^{(n-1)} & L^{(n-1)}Q \\ -P^T & 2^{n-1} - P^T Q \end{pmatrix}$$

Let us consider what each term is. $L^{(n-1)}$ is just the $(n-1)\times(n-1)$ matrix of the form:

$$\begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ -1 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \\ -1 & -1 & -1 & \cdots & 1 \end{pmatrix}$$

We already know what $-P$ is. $L^{(n-1)}Q$ gives:

$$\begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -1 & 1 & 0 & \cdots & 0 \\ -1 & -1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \\ -1 & -1 & -1 & \cdots & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 4 \\ \vdots \\ 2^{n-2} \end{pmatrix} = \begin{pmatrix} 1 \\ 2-1 \\ 4-2-1 \\ \vdots \\ 2^{n-2} - \sum_{i=0}^{n-3} 2^i \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

$2^{n-1} - P^T Q$ gives:

$$2^{n-1} + \begin{pmatrix} -1 & -1 & \cdots & -1 \end{pmatrix}^T \begin{pmatrix} 1 \\ 2 \\ 4 \\ \vdots \\ 2^{n-2} \end{pmatrix} = 2^{n-1} - \sum_{i=0}^{n-2} 2^i = 1$$

Therefore $L^{(n)}U^{(n)} = A^{(n)}$, and we have the desired result (since LU factorisations with 1's on diagonal are unique).

1(c)

First, we examine the error of the **"LUP_inplace"** algorithm for A^{60}, which has a growth factor of 2^{59}. If the algorithm is backward stable, then there exists there exists a perturbed input $A^{(60)} + \delta A^{(60)}$ such that $\frac{\|\delta A^{(60)}\|}{\|A^{(60)}\|} = \mathcal{O}(\epsilon)$, where $\epsilon \approx 1.11 \times 10^{-16}$ is

the machine epsilon, and $\tilde{L}\tilde{U}$ is the solution to the problem, i.e. $A^{(60)} + \delta A^{(60)} = \tilde{L}\tilde{U}$. This would imply: $\frac{\|A^{(60)} - \tilde{L}\tilde{U}\|}{\|A^{(60)}\|} = \mathcal{O}(\epsilon)$. The file **"Test_A60.py"** contains code that calculates this, and we see that $\frac{\|A^{(60)} - \tilde{L}\tilde{U}\|}{\|A^{(60)}\|} \approx 0.05636 >> \epsilon$, which shows that the algorithm is unstable.

Next, we examine the stability of the algorithm **"solve_LUP"**. **"Test_A60.py"** contains the code that does this. We first generate a random vector x, compute the product $b = A^{(60)}x$, apply **"solve_LUP"** to get a perturbed output $\tilde{x}$, then measure the error of the algorithm, which is quantity $\frac{\|b - \tilde{b}\|}{\|b\|}$ ($\tilde{b} = A^{(60)}\tilde{x}$). We do so 20 times and obtain $0.3667 >> \epsilon$ as an approximate mean for error. The following plot demonstrates the rough pattern of the errors:

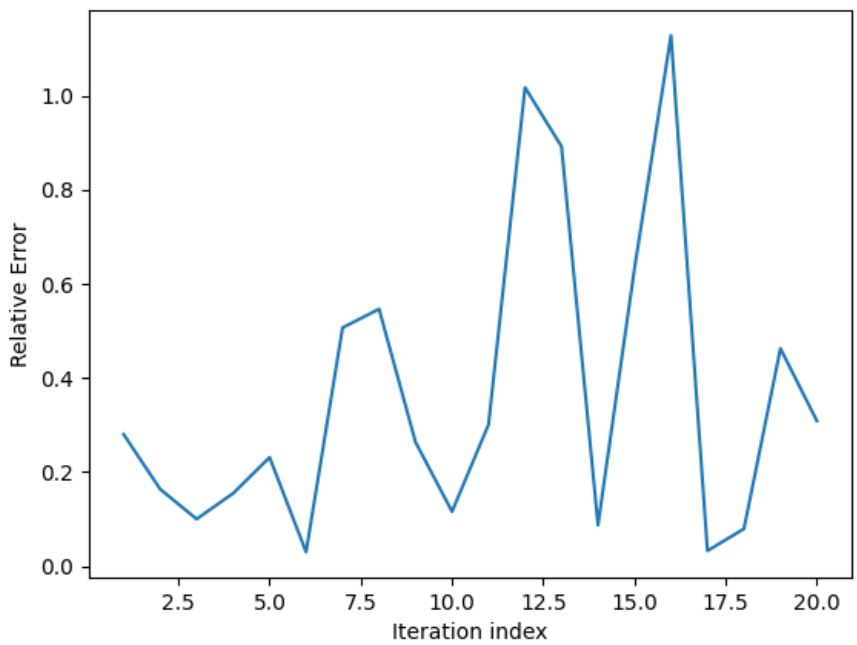

Figure 1: Relative Error for solve LUP with A^{60}

As we can see, the error $>> \epsilon$. Hence we conclude that **"solve_LUP"** is also not stable.

1(d)

For this part, we will calculate the average error of the **"solve_LUP"** algorithm for matrices of size $N = 5, 10, \ldots, 100$, whose entries are independently distributed random variables drawn from the uniform distribution on the interval $\left[-\frac{1}{N}, \frac{1}{N}\right]$. For each dimension N, we apply **"LUP_inplace"** to get the LU decomposition for the matrix A, then measure the growth factor using the formula from lecture notes. We repeat

this procedure twice and measure the average to reduce the chance of getting outlier values for each dimension of the matrix. The following plot demonstrates the rough pattern for growth factor as N becomes large (code for plot can be found in **Q1d.py**):

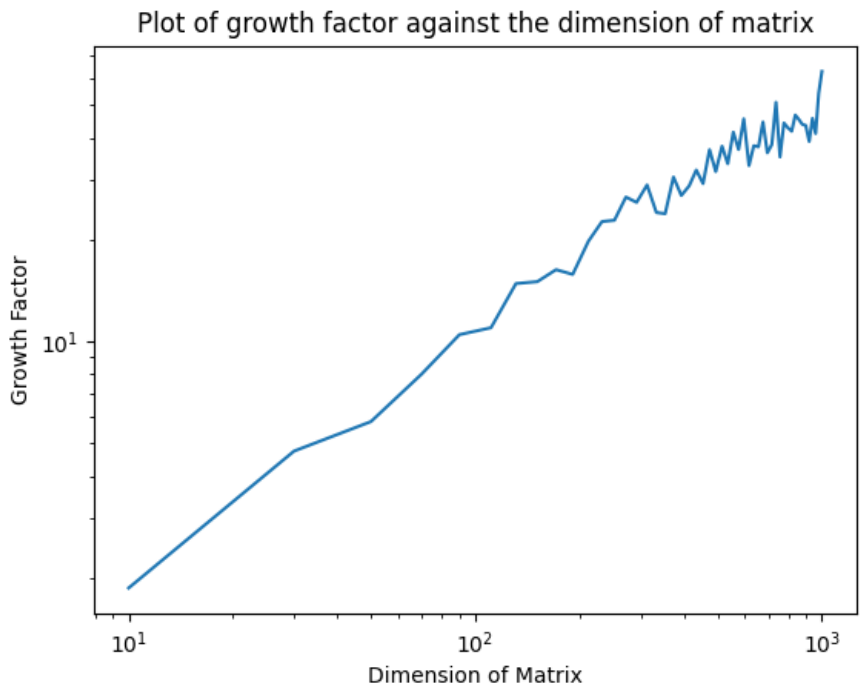

Figure 2: Relative Error for solve LUP with varying matrix dimensions

This is a loglog plot of the growth factor against N. As we can observe from the graph, the graph is roughly linear, which means that the growth factor grows at a rate that is of some power growth rate. This is good news; in the previous worst case scenario, growth factor grows exponentially (2^{N-1}) instead of by power; power growth is much slower than exponential growth. Therefore, in general cases, the growth factor for large N should be a lot lower than the worst case scenario 2^{N-1}, which means that the algorithm is relatively stable.

Question 2

2(a)

The associated function can be found under the **"cw2"** directory in **"Q2.py"**. It implements the algorithm as described by the question. Tests are provided under **"test"** directory in the file **"test_CW2_Q2.py"**. The tests test the function by generating random matrix A and random vector x, form the product $b = Ax$, then use **"MGS_solve_ls"** to compute a numerical solution x_1 for $Ax = b$. We verify that $\|x_1 - x\|$ is within a certain threshold of accuracy (10^{-6}).

2(b)

For this question, we generate a random vector x^*, obtain the product $b = Ax^*$, then obtain numerical solutions $\hat{x}_1$ and $\hat{x}_2$ for the system $A\hat{x} = b$, by using **"MGS_solve_ls"** and **"householder_ls"** methods. Then, we compute $\|\hat{x}_i - x^*\|$ for $i = 1, 2$ to calculate the errors associated with each method. We repeat the above procedure 100 times with different x^*, and relevant code can be found in **"Q2b.py"**. The following plot shows how the errors produced for both MGS method and householder method look; note that in the graph, we have taken log_{10} of the error values, since the Householder errors are significantly smaller than the MGS errors and taking log allows better comparison.

Figure 3: Comparison of error between Housholder and MGS methods

From the graph, we can see that for the matrix A_2, errors produced by MGS method is approximately $\mathcal{O}(10^{-6})$ whereas errors produced by Householder method is around $\sim \mathcal{O}(10^{-11})$. We can conclude that **"MGS_solve_ls"** is significantly less accurate than **"householder_ls"**.

2(c)

We have that $A_+ = (A|b) = \underbrace{\begin{pmatrix} \hat{Q} & q_{n+1} \end{pmatrix}}_{Q} \underbrace{\begin{pmatrix} \hat{R} & z \\ 0 & \rho \end{pmatrix}}_{R}$. By multiplying out the matrix in block form and equating the terms, we can immediately see that $b = \hat{Q}z + \rho q_{n+1}$. To see

what value z takes, left multiply by Q^T. Then we have: $\underbrace{\begin{pmatrix} \hat{Q}^T \\ q_{n+1}^T \end{pmatrix}}_{Q^T} \begin{pmatrix} A & b \end{pmatrix} = \underbrace{\begin{pmatrix} \hat{R} & z \\ 0 & \rho \end{pmatrix}}_{R}$.

($Q^T Q = I$ so Q on RHS disappears). We can see that $z = \hat{Q}^T b$. Hence we have:

$$z = \hat{Q}^T b$$
$$b = \hat{Q} z + \rho q_{n+1}$$

2(d)

Suppose that A is of dimension $m \times n$, we propose the following alternative algorithm:

Modified MGS

- $A_+ \leftarrow (A \mid b)$ (STEP 1)
- Obtain $R = \begin{pmatrix} \hat{R} & z \\ 0 & \rho \end{pmatrix}$ from QR factorisation of A_+ (STEP 2)
- $z \leftarrow R_{1:n,n+1}$ (STEP 3)
- $\hat{R} \leftarrow R_{1:n,1:n}$ (STEP 4)
- $x = \text{solve_U}(\hat{R}, z)$ (STEP 5)

Let's analyse the operation count of each step. In STEP 1, we are adding an extra column to the right of A. This should be $\mathcal{O}(n)$ operations, since there are n entries in b. In STEP 2, we are calculating the QR factorisation of a $m \times (n+1)$ matrix. From lecture notes, we know that this is $\mathcal{O}(2m(n+1)^2)$ operations. In STEP 3, we are slicing out z from A_+. This should be $\mathcal{O}(n)$ operations, since z is a vector of length n. In STEP 4, we are slicing out $\hat{R}$, a $n \times n$ matrix, which should be $\mathcal{O}(n^2)$ operations. Lastly, in STEP 5, we are applying the backward substitution algorithm. From lecture notes, we know that this is $\mathcal{O}(n^2)$ operations. Therefore, we can see that STEP 2 is the dominant algorithm in terms of operation counts. Hence, the total operation count for **Modified MGS** should be: $\approx \mathcal{O}(2m(n+1)^2 + 2n^2) \approx \mathcal{O}(mn^2)$

Automated tests are also provided in the **test** folder. The tests follow the same procedure as the tests from 2(a).

2(e)

We follow a similar procedure to part 2(b) to compare the errors. We generate a random vector x^*, obtain the product $b = Ax^*$, then obtain numerical solutions $\hat{x}_1$ and $\hat{x}_2$ for the system $A\hat{x} = b$, by using **"MGS_solve_ls"** and **"MGS_solve_ls_modified"** methods. Then, we compute $\|\hat{x}_i - x^*\|$ for $i = 1, 2$ to calculate the errors associated with each method. We repeat the above procedure 100 times with different x^*, and relevant code can be found in **"Q2e.py"**. The following plot shows what errors

produced by each method look like. Note that in the graph, we have taken log_{10} of the error values, since the errors associated with augmented A_+ are significantly smaller than the normal MGS errors.

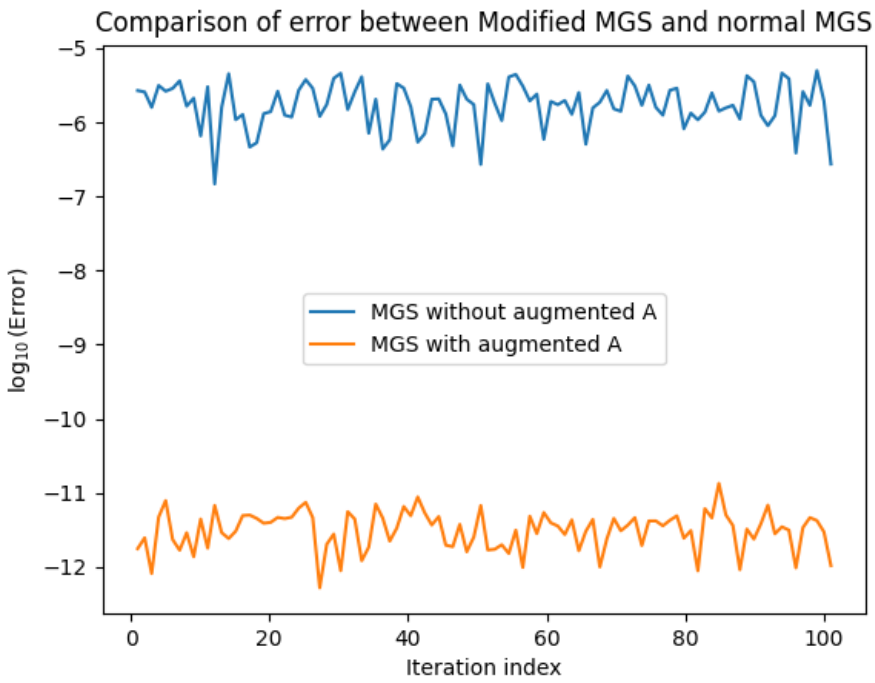

Figure 4: Comparison of errors between Modified_MGS and normal MGS

From this graph, we can see that using the matrix A_+ greatly increases the accuracy, since the errors produced by the unmodified MGS method is around $\mathcal{O}(10^{-6})$, whereas errors produced by the modified MGS method is around $\mathcal{O}(10^{-11}) \sim \mathcal{O}(10^{-12})$.

2(f)

For this question, we want to generate $b = Ax^* + r$ which is outside of the column range of A. If b is outside of A's column range, then r must also be outside of A's column range. To generate such r, we first randomly generate a vector v and append it to the end of A to form the augmented matrix $\hat{A}$. Then, we perform QR factorisation on $\hat{A}$; we take r to be the last column of the matrix $\hat{Q}$ from the factorisation. To see why r is outside of A's column range, we consider the nature of QR factorisation; the span of the first n columns of $\hat{Q}$ is equal to the span first n columns of $\hat{A}$, which is essentially the column space of A. Moreover, columns of $\hat{Q}$ are orthogonal to each other. Hence, the $(n+1)$-th column of $\hat{Q}$ is orthogonal to all other columns of $\hat{Q}$, and therefore must be outside of A's column space.

After we determine r, we can now generate x^* and form $b = Ax^* + r$. We compute the numerical solutions $\hat{x}_1$, $\hat{x}_2$, $\hat{x}_3$ for the system $A\hat{x} + r = b$, using **"MGS_solve_ls"**, **"MGS_solve_ls_Modified"** and **"householder_ls"** methods. Then, we compute $\|\hat{x}_i -$

$x^*\|$ for $i = 1, 2, 3$ to calculate the errors associated with each method. We repeat the above procedure 100 times with different x^* and r; the relevant code can be found in **"Q2f.py"**. We will still use log_{10} scales for errors, so that we can compare the errors better. The following plot is generated:

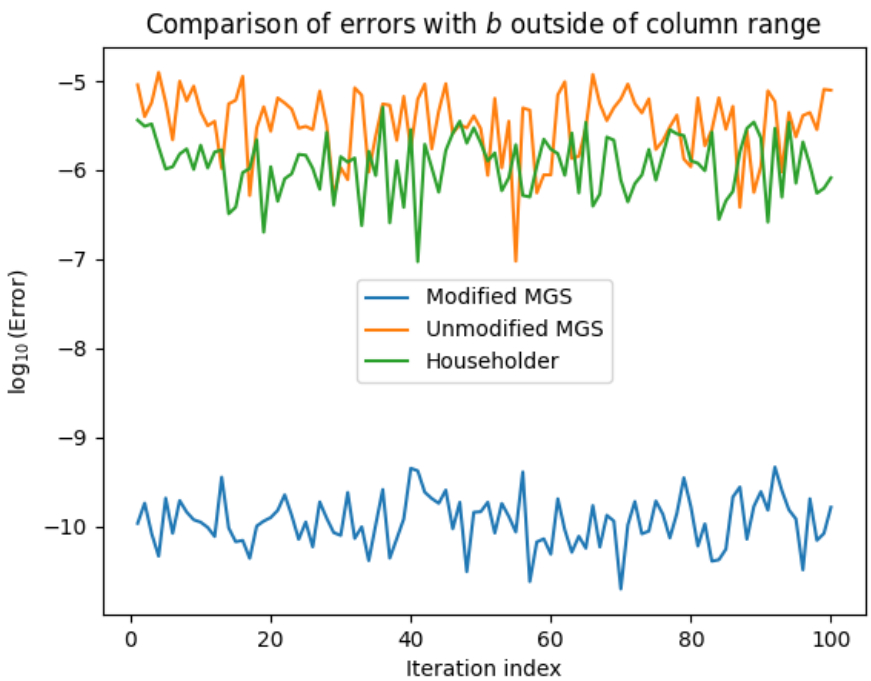

Figure 5: Comparison of errors with b outside of column range

From the plot, we can see that modified MGS still massively outperforms the other methods, as the error associated with modified MGS is roughly $\mathcal{O}(10^{-10})$, whereas errors produced unmodified MGS and householder are around $\mathcal{O}(10^{-5}) \sim \mathcal{O}(10^{-6})$.

Question 3

3(a)

The formula for converting from the array u to the vector $\hat{u}$ is given by:

$$\hat{u}_{(i-1)N+j} = u_{ij} \tag{4}$$

The form of matrix D is the following:

$$
D = \begin{bmatrix}
\overbrace{\begin{matrix}
1+4s^2 & -s^2 & 0 & \cdots & 0 \\
-s^2 & \ddots & \ddots & & \vdots \\
\vdots & \ddots & \ddots & \ddots & \vdots \\
\vdots & & \ddots & \ddots & -s^2 \\
0 & \cdots & \cdots & -s^2 & 1+4s^2
\end{matrix}}^{N \times N \text{ block}} & \begin{matrix} -s^2 \\ \\ \\ \\ 0 \end{matrix} & & & & & & & & & & & & & \\
\begin{matrix} -s^2 \\ \\ \\ \\ 0 \end{matrix} & \begin{matrix} 1+4s^2 \\ -s^2 \\ \vdots \\ 0 \end{matrix} & \begin{matrix} -s^2 \\ \cdots \\ \\ \\ \end{matrix} & & & & & & & & & & & & \\
& \begin{matrix} -s^2 \\ \\ \\ \\ \end{matrix} & \begin{matrix} -s^2 \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & & & & & & & & \\
& & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & & & & & & \\
& & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & & & & & \\
& & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & & & & \\
& & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & & & \\
& & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & & \\
& & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & & \\
& & & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & & \\
& & & & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \\
& & & & & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} -s^2 \\ \ddots \\ \ddots \\ \\ \end{matrix} \\
& & & & & & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} \\
& & & & & & & & & & & & \begin{matrix} \ddots \\ \ddots \\ \ddots \\ \\ \end{matrix} \\
& & & & & & & & & & & & \begin{matrix} -s^2

3(b)

In the files **"exercises2.py"**, **"exercises6.py"**, **"exercises7.py"**, the parameters "bl, bu" are added to the functions **"solve_L"**, **"solve_U"**, **"solve_LU"** and **"LU_inplace"** for banded matrices. The tests for these functions can be found under the directory **test** in the file **"test_CW2_Q3.py"**. The tests check if the functions work as intended for banded matrices when the parameters "bl, bu" are used during the function call.

3(c)

The scripts for the plot can be found in the file **Q3c.py**, and the functions used can be found in **Q3.py**. The function **Mat_D** takes in parameters N, s, and generates matrix D according to the provided parameters. The function **sim** simulates a heatmap for the solved u-array. We generate the following plots for $N = 50$ and varying values of s:

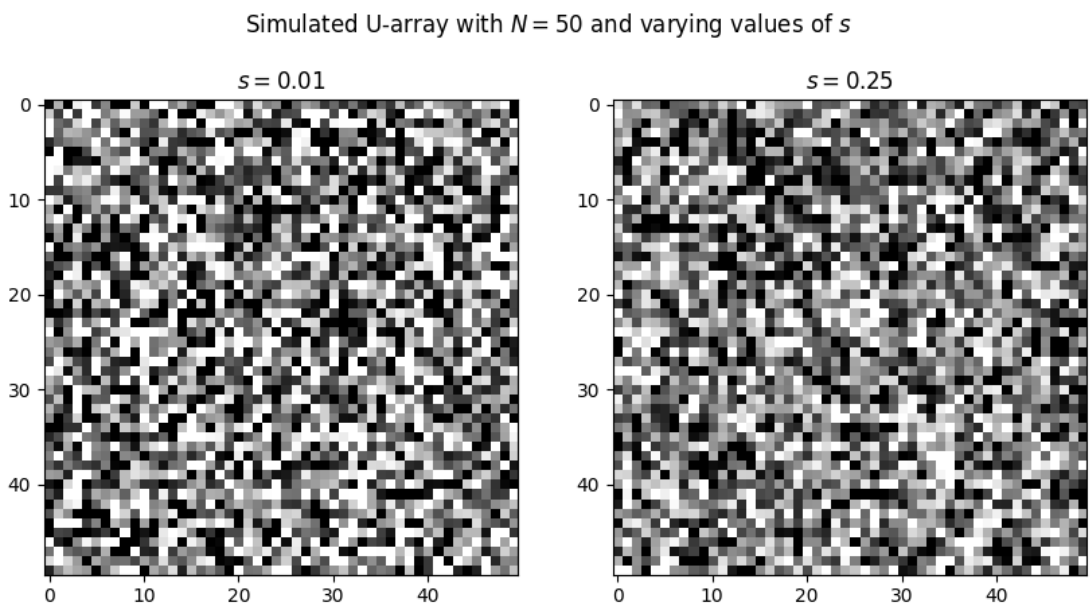

Figure 6: Heatmap for $N = 50$, $s = 0.01, 0.25$

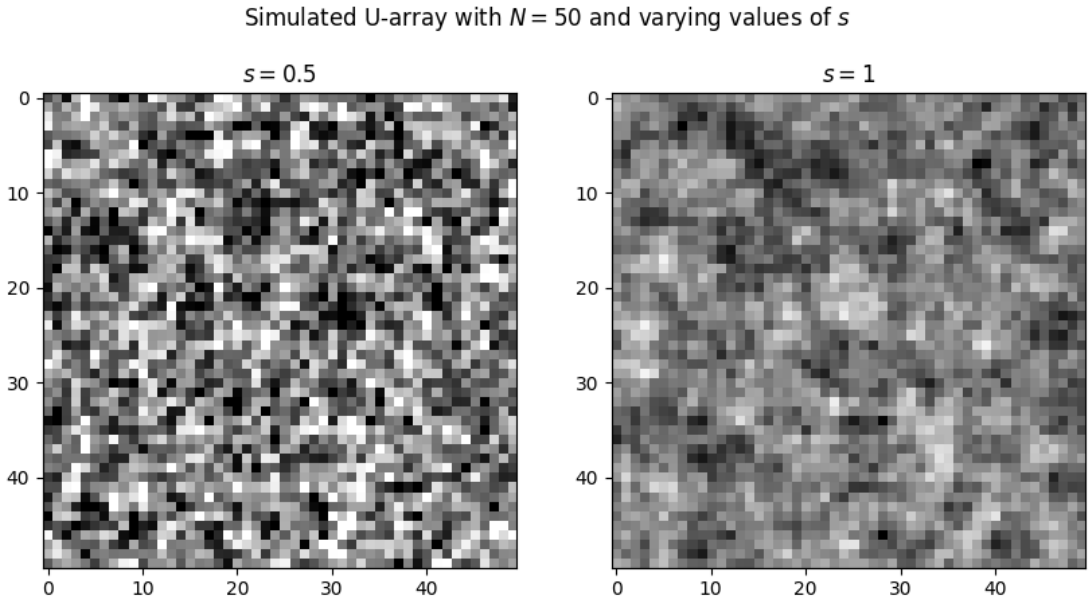

Figure 7: Heatmap for $N = 50$, $s = 0.5, 1$

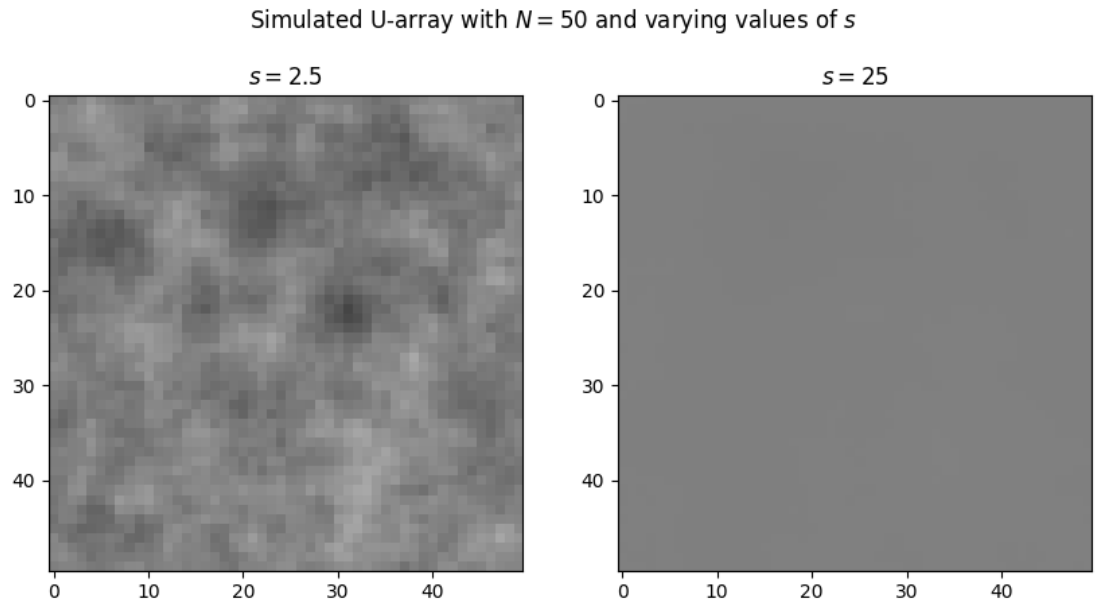

Figure 8: Heatmap for $N = 50$, $s = 2.5, 25$

In our plots, we have generated the heatmaps using the same color range, so that we can compare the spread of values in U-array on the same scale.

From the figures, we can see when s is small, the image is very pixelated, indicating that there is a comparatively big spread of u_{ij} values. When we increase s, the image tends towards the same colour, which indicates that u_{ij}'s are tending towards the same value. Indeed, when s is small, the correlation between adjacent entries is also very small and matrix $D \approx I$. This means that the distribution of entries of the U-

array would tend towards a multivariate normal distribution with mean 0 and $\sum = I$ as the covariance matrix (since $\hat{u} \sim (0, (D^2)^{-1})$. When s becomes large, the values of U-array tend towards the same value, which also makes sense as adjacent values are more strongly correlated to each other when s becomes large.

3(d)

We would expect the runtime for banded **"solve_LU"** to be significantly lower than the original solver. To see why that is, let us consider the operation count for each algorithm. The original algorithm has complexity $\mathcal{O}(m^3)$, as stated in the lecture notes (m is the dimension of the matrix). In our context, the matrix D is of dimension N^2, hence the original algorithm would have complexity $\mathcal{O}(N^6)$. For banded matrices, solving the system requires $\mathcal{O}(mpq)$ operations, where m is the dimension of the matrix, p the lower bandwidth and q the upper bandwidth. In our context, $m = N^2$, $p = q = N$; hence the banded solver has complexity $\mathcal{O}(N^4)$. We generate the following graph of solver runtimes for different values of N (the code can be found in file **Time_Test.py**):

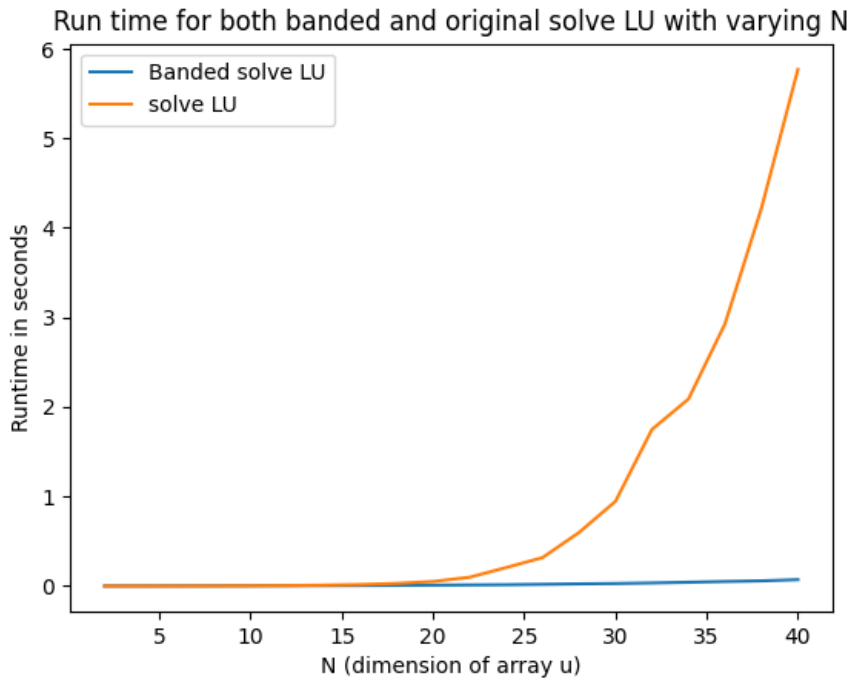

Figure 9: Runtime of solvers for different N

We can see that, the run time for original **"solve_LU"** blows up a lot faster than the banded version, which confirms our hypothesis, since N^6 also blows up a lot faster than N^4 as N becomes bigger.

3(e)

We have:

$$((1+\rho)I + s^2 H)\hat{u}^{n+\frac{1}{2}} = (\rho I - s^2 V)\hat{u}^n + w \tag{5}$$

$$((1+\nu)I + s^2 V)\hat{u}^{n+1} = (\nu I - s^2 H)\hat{u}^{n+\frac{1}{2}} + w \tag{6}$$

We first claim that $I + s^2(H+V) = D$. To see why that's the case, we consider the forms of matrices H and V. V is a block tridiagonal matrix, with N^2 blocks of identical $N \times N$ blocks V_i, defined as:

$$V_i = \underbrace{\begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & -1 \\ 0 & \cdots & \cdots & -1 & 2 \end{bmatrix}}_{N \times N \text{ block}} \tag{7}$$

H is the matrix with 2's on the main diagonal and -1's on the n-th diagonal above and below the main diagonal, i.e.

$$H = \begin{bmatrix} 2 & \cdots & -1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ -1 & \ddots & \ddots & \ddots & -1 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & -1 & \cdots & 2 \end{bmatrix}$$

We can see by inspection that $I + s^2(H+V) = D$.

Now, suppose that $\hat{u}$ converge to a limit $\hat{u}^*$. Then, we have:

$$((1+\rho)I + s^2 H)\hat{u}^{*+\frac{1}{2}} = (\rho I - s^2 V)\hat{u}^* + w \tag{8}$$

$$((1+\nu)I + s^2 V)\hat{u}^* = (\nu I - s^2 H)\hat{u}^{*+\frac{1}{2}} + w \tag{9}$$

Upon rearranging 8 we can obtain:

$$w = ((1+\rho)I + s^2 H)\hat{u}^{*+\frac{1}{2}} - (\rho I - s^2 V)\hat{u}^* \tag{10}$$

Substituting 10 into 9 we obtain:

$$\begin{aligned} ((1+\nu)I + s^2 V)\hat{u}^* &= (\nu I - s^2 H)\hat{u}^{*+\frac{1}{2}} + ((1+\rho)I + s^2 H)\hat{u}^{*+\frac{1}{2}} - (\rho I - s^2 V)\hat{u}^* \\ (1+\nu+\rho)I\hat{u}^* &= (1+\nu+\rho)I\hat{u}^{*+\frac{1}{2}} \\ \hat{u}^* &= \hat{u}^{*+\frac{1}{2}} \end{aligned}$$

This shows that the intermediate variable $\hat{u}^{n+\frac{1}{2}}$ also converges to the same limit. Substituting $\hat{u}^* = \hat{u}^{*+\frac{1}{2}}$ into 8 we obtain:

$$\begin{aligned}((1+\rho)I + s^2 H)\hat{u}^* &= (\rho I - s^2 V)\hat{u}^* + w \\ (I + s^2(H+V))\hat{u}^* &= w \\ D\hat{u}^* &= w\end{aligned}$$

which is the required form.

3(f)

In the banded algorithm, the operation count is $\mathcal{O}(N^4)$. This is because we are performing Gaussian elimination on a $N^2 \times N^2$ matrix with both lower bandwidth and upper bandwidth N. From lecture notes, we know that this algorithm has complexity $\mathcal{O}(mpq)$, where m is the dimension of the matrix, p the lower bandwidth and q the upper bandwidth. In our case, $m = N^2, p = N, q = N$, hence the $\mathcal{O}(N^4)$ complexity. We propose an alternative algorithm that solves 5, 6 in $\mathcal{O}(N^2)$ operations.

We first introduce a permutation that permutes the system $Ax = b$ to $A'x' = b'$, where A is a band-N, $N^2 \times N^2$ matrix that has the same value α on its main diagonal and the same value λ on both its offset diagonals. x, b are arbitrary length N^2 vectors. Moreover,

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_N \\ x_{N+1} \\ x_{N+2} \\ \vdots \\ x_{N^2} \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_N \\ b_{N+1} \\ b_{N+2} \\ \vdots \\ b_{N^2} \end{pmatrix}, \quad x' = \begin{pmatrix} x_1 \\ x_{N+1} \\ x_{2N+1} \\ \vdots \\ x_{(N-1)N+1} \\ x_2 \\ x_{N+2} \\ \vdots \\ x_{N^2} \end{pmatrix}, b' = \begin{pmatrix} b_1 \\ b_{N+1} \\ b_{2N+1} \\ \vdots \\ b_{(N-1)N+1} \\ b_2 \\ b_{N+2} \\ \vdots \\ b_{N^2} \end{pmatrix}$$

The modified matrix A' that satisfies $A'x' = b'$ is a $N^2 \times N^2$ block diagonal matrix with N tridiagonal blocks A_i where A_i is:

$$A_i = \begin{pmatrix} \alpha & \lambda & 0 & \cdots & 0 \\ \lambda & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \lambda \\ 0 & \cdots & \cdots & \lambda & \alpha \end{pmatrix}$$

To get intuition to why that is the case, let us consider the structure of the original system: when computing the i-th component of vector b, we only receive contribution from the i-th component of x and the two components that are N indices away

from x_i. Therefore, it makes sense to sort the indices with respect to modulo N, so that the components that contribute to each other's calculations are adjacent. The indices of a N^2 length vector can be sorted into cosets of modulo N with each coset containing N elements, which explains why modified A' is composed of N blocks of $N \times N$ matrices. The tridiagonality of the blocks arises from the fact indices that were "distance" N apart from each other in x are now only "distance 1" apart in the permuted x_i.

To solve the modified system, instead of applying **"solve_LU"** to the whole matrix, we can instead solve the system by blocks. We have N blocks to solve in total. Since each block is of size $N \times N$ and band 1, solving one block would require complexity $\mathcal{O}(N \times 1 \times 1) = \mathcal{O}(N)$. There are N blocks in total, hence the complexity for solving the whole system is $\mathcal{O}(N^2)$.

The algorithm for solving the system $Ax = b$ is as follows (where A, x and b are as described above):

Algorithm 1

- $x \leftarrow 0^{N^2}$ (zero vector of length N^2)
- Apply permutation to vector b to form b'
- Form one block A_i of the modified matrix A'
- FOR $i = 1$ TO N:
  - $x'_{iN+1:(i+1)N} = \text{solve_LU}(A_i, b'_{iN+1:(i+1)N}, bl = 1, bu = 1)$ (Using the banded solver with bandwidth 1 for both lower and upper bandwidths)
  - END FOR
- Invert the permutation on x' to obtain x
- END FOR

Note that if we apply **Algorithm1** to H from equations 5 and 6, the modified H' has the exact same form as V. In the system of equations 5 and 6, all matrices involved in the calculations are just linear combinations of H, V and I. When applying **Algorithm1** to a matrix A, the main diagonal's entries of A are unchanged. Therefore, if we apply the transformation entailed by **Algorithm1** to matrices that are in the form $aH + bI$, these matrices can also be written in block diagonal form, where each block is tridiagonal and has the form $aV_i + bI$.

Now, let us propose an alternative algorithm that computes matrix-vector product which takes the bandedness of the matrix into account:

Algorithm 2

- $b \leftarrow 0^N$ (zero vector of length N)

- FOR $i = 1$ TO N:
  - $j = min(i + bandwidth,\ N)$
  - $k = max(1,\ i - bandwidth)$
  - FOR $l = j$ TO k:
    - $b_i \leftarrow b_i + A_{il} x_l$
  - END FOR
- END FOR

For a $N \times N$, band-m matrix, this has complexity $\mathcal{O}(mN)$, whereas the standard matrix vector product has complexity $\mathcal{O}(N^2)$.

Combining the **"Algorithm1"** and **"Algorithm2"**, we can now solve 5, 6 using the following procedure:

Algorithm 3

- Generate V_i as in equation 7
- Generate the following $N \times N$ matrices:
  - $A_1 = (1+\rho)I + s^2 V_i$ (the $N \times N$ block from the block diagonal matrix $(1+\rho)I + s^2 H'$, where H' is the modified H matrix)
  - $A_2 = (1+\nu)I + s^2 V_i$ (the $N \times N$ block from the block diagonal matrix $(1+\nu)I + s^2 V$)
  - $A_3 = \rho I - s^2 V_i$ (the $N \times N$ block from the block diagonal matrix $\rho I - s^2 V$)
  - $A_4 = \nu I - s^2 V_i$ (the $N \times N$ block from the block diagonal matrix $\nu I - s^2 H'$, where H' is the modified H matrix)
- Generate vector w as described by the question
- $\hat{u} \leftarrow 0^{N^2}$ (zero vector of length N^2)
- Generate P, P^{-1} (The permutation and its inverse, stored as vectors of indices)
- FOR $t = 1, 2, 3 \ldots$
  - $v_1 \leftarrow 0^{N^2}$ (zero vector of length N^2 for storing outputs)
  - $v_2 \leftarrow 0^{N^2}$ (zero vector of length N^2 for storing outputs)
  - $\hat{u}^* \leftarrow 0^{N^2}$ (zero vector of length N^2 for storing outputs)
  - FOR $i = 0$ TO $N - 1$ (**STEP 1**):
    - $(v_1)_{iN+1:(i+1)N} \leftarrow$ **Algorithm2**$(A_3, \hat{u}_{iN+1:(i+1)N})$ (Multiplying out A_3 and $\hat{u}$ block-wise)
    - END FOR

- $v_1 \leftarrow \; v_1 + w$ (Add vector w)
- $v_1 \leftarrow \; Pv_1$ (Permute v_1)
- FOR $j = 0$ TO $N-1$ (**STEP 2**):
  * $\hat{u}^*_{jN+1:(j+1)N} \leftarrow$ **Algorithm1**$(A_1, (v_1)_{[jN+1:(j+1)N]})$ (Solving for $\hat{u}^*$ block-wise for each A_1 block)
  * END FOR
- FOR $k = 0$ TO $N-1$ (**STEP 3**):
  * $(v_2)_{kN+1:(k+1)N} \leftarrow$ **Algorithm2**$(A_4, \hat{u}^*_{kN+1:(k+1)N})$ (Multiplying out A_4 and $\hat{u}^*$ block-wise)
  * END FOR
- $v_2 \leftarrow P^{-1}v_2 + w$ (Invert the permutation on v_2 and add w)
- FOR $l = 0$ TO $N-1$ (**STEP 4**):
  * $\hat{u}_{lN+1:(l+1)N} \leftarrow$ **Algorithm1**$(A_2, (v_2)_{[lN+1:(l+1)N]})$ (Solving for $\hat{u}$ block-wise for each A_2 block)
  * END FOR

Whilst this algorithm looks really long, each iteration requires only $\mathcal{O}(N^2)$ operations, since **"Algorithm1"** and **"Algorithm2"** are both $\mathcal{O}(N^2)$ and assigning/permuting values in arrays with N^2 entries or less also only require $\mathcal{O}(N^2)$ operations. Moreover, implementing this algorithm is more efficient memory wise than solving $D\hat{u} = w$. This is because solving for $D\hat{u} = w$ using the banded solver requires us to create the matrix D, which is of dimension $N^2 \times N^2$, whereas in **"Algorithm 3"**, we are only storing $N \times N$ matrices and length N^2 vectors, which requires much less memory.

3(g)

The algorithm is implemented in the file **"Q3.py"** in the function **"iterative_solver"**. An optional parameter "test" (that has default value False) is used for testing. If "test = True", then the function will also return w. Automated tests are provided in **"test_CW2_Q3.py"**, where we form matrix D and check that $\|D\hat{u} - w\| < 10^{-6}\|w\|$.

3(h)

In the figure below, we generate each graph by keeping ν, s fixed and vary N and ρ. We do so 5 times for 5 different values of ν, namely $0.1, 0.5, 1, 1.5, 2$. We generate the following graphs:

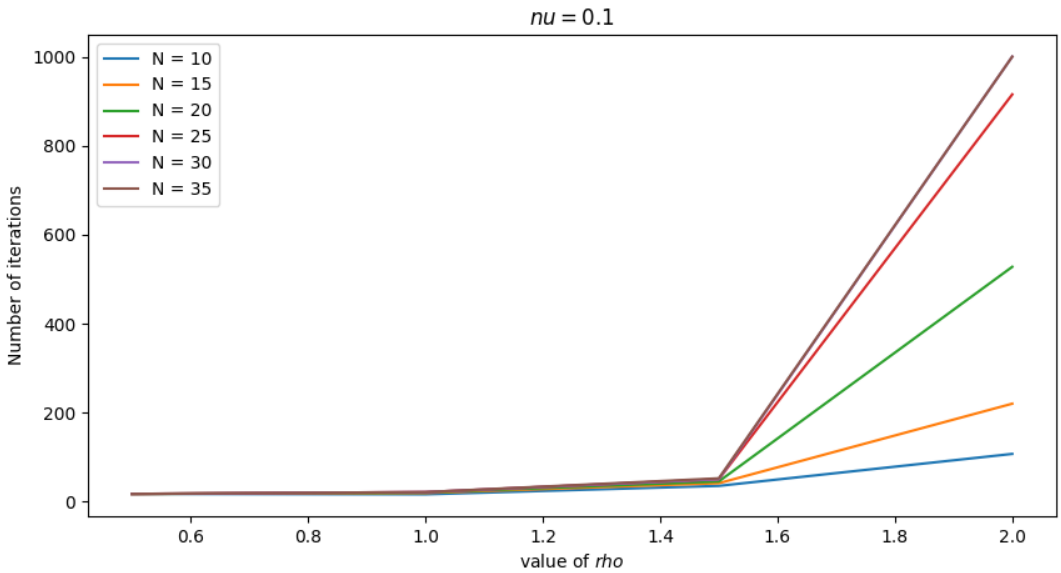

Figure 10: Iteration counts when $\nu = 0.1$

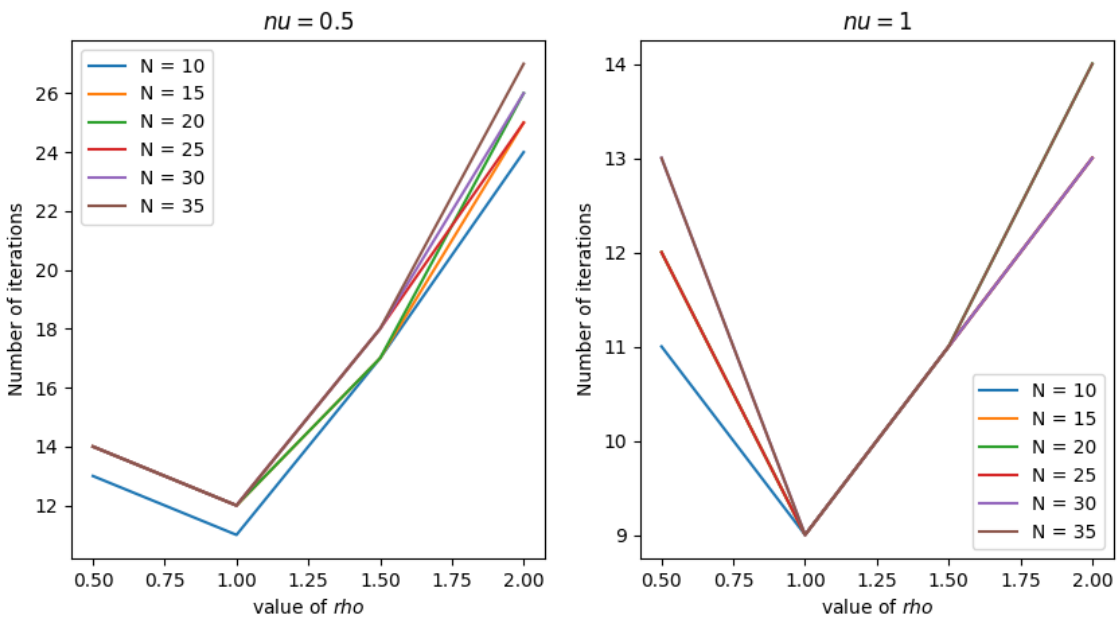

Figure 11: Iteration counts when $\nu = 0.5, 1$

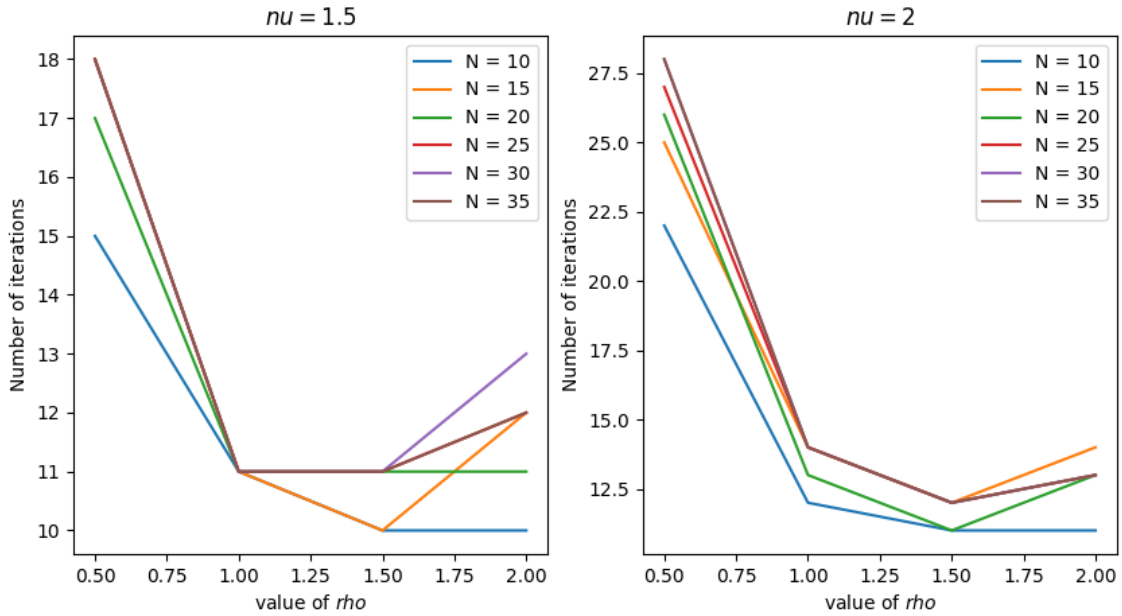

Figure 12: Iteration counts when $\nu = 1.5, 2$

From inspecting the figures, we can see that in most cases, the solution converges fastest when we take $\nu = 1, \rho = 1$. For all the tested cases, the solutions converge in 9 iterations every time, independent of the dimension of the matrix. In terms of operation counts, this means that the typical operation count should still be $\mathcal{O}(N^2)$, since the rate of convergence is independent of the size of the matrix.

3(i)

On a parallel computer, one can solve blocks of equations simultaneously instead of iteratively. This can greatly speed up **STEP 2** and **STEP 4** in the algorithm, since the algorithm is only solving one system of equations at a time.